

When a Machine Learns to Create

Supervised, Unsupervised and Generative AI · AI and Machine Learning · Week 2, Day 5 (Final Day)

Where we left off

We taught machines to see and to read. Today: how they learn, and how they create.

How models learn

With labels, or from raw data alone.

Tell apart vs imagine

Discriminative models sort; generative models create.

Generative AI

The tools behind ChatGPT and AI art.

From recognising the world, to creating brand new images, text and ideas.

The last step of our journey, and the deepest: the ideas powering today's generative AI.

Part 1

Two Ways a Machine Learns

Supervised learning: learn from labelled examples



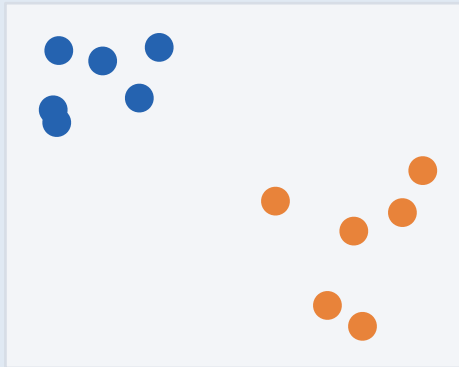
Data comes in pairs: an input x and its correct answer y . The model learns to map x to y .



Examples: classification, regression, object detection and segmentation. All need labelled data.

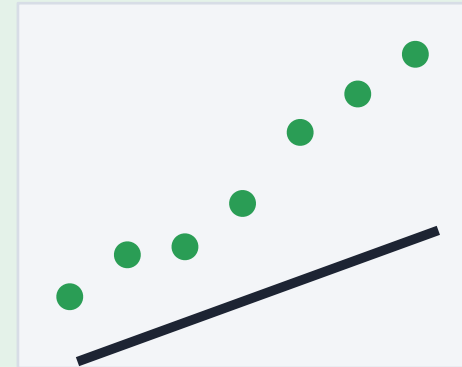
Two flavours of supervised learning

Classification



Predict a category: cat or dog, spam or not.

Regression



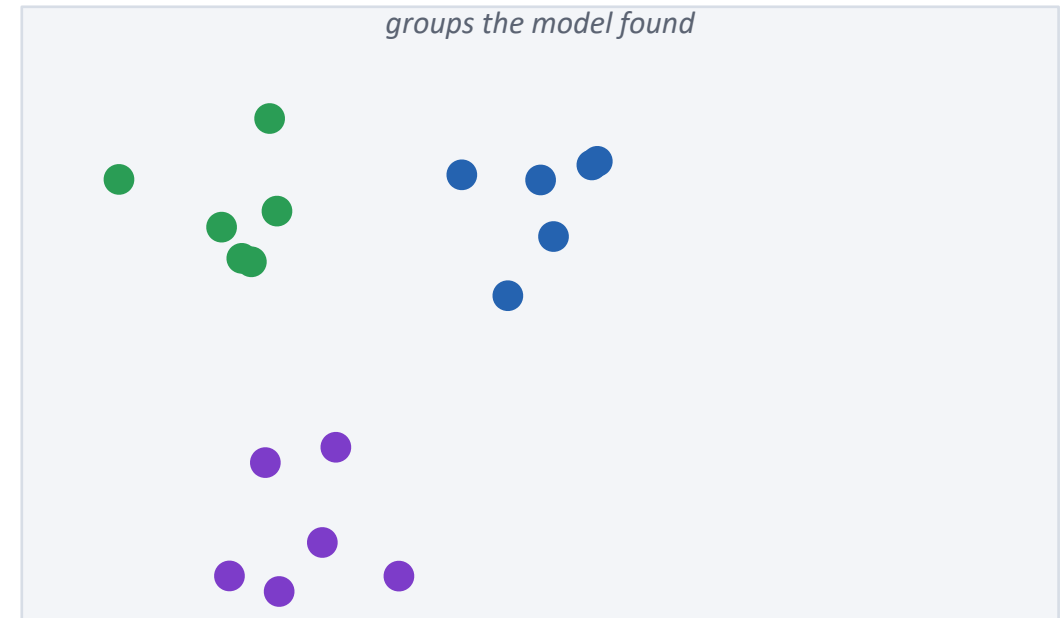
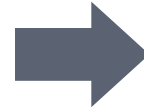
Predict a number: price, yield, temperature.

Same setup, different output: a label for classification, a continuous value for regression.

Unsupervised learning: find structure without labels



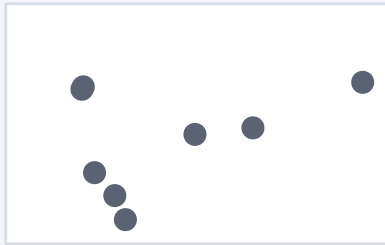
Just data, no labels. The model discovers the hidden structure on its own.



Examples: clustering, dimensionality reduction and feature learning. No answer key required.

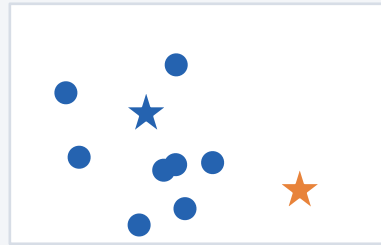
Clustering up close: K-means

1 Place centres



Drop K markers at random.

2 Assign



Each point joins its nearest centre.

3 Move and repeat

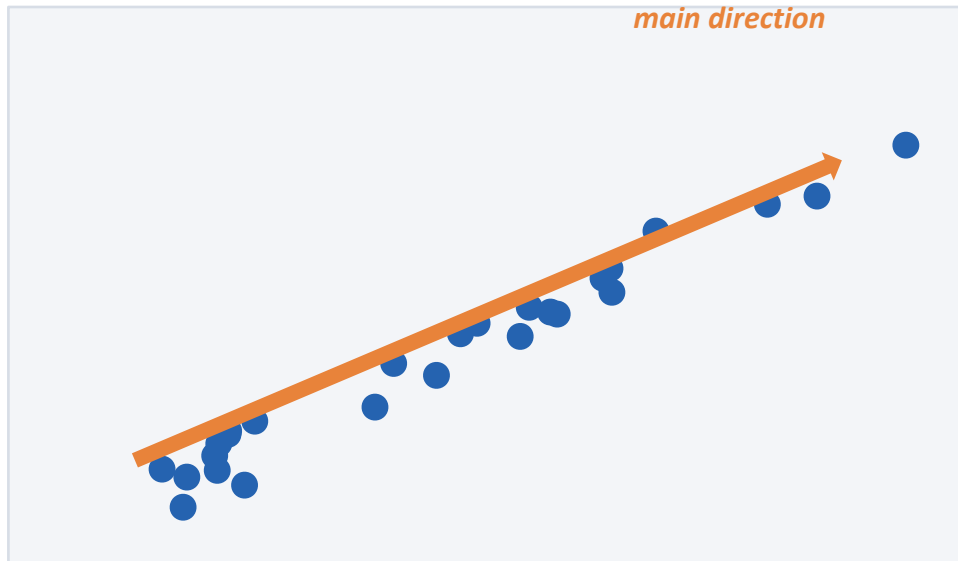


Shift centres to the middle, repeat.

Repeat assign and move until the groups settle. You choose K, the number of clusters.

Dimensionality reduction: keep what matters

Squeeze many features down to a few, while keeping most of the variation. PCA finds the main directions.



Project onto the main direction and most of the spread is kept in far fewer numbers.

Useful for visualising data in 2D and for speeding up other models.

Fewer dimensions means simpler models and pictures we can actually look at.



Supervised vs unsupervised

Supervised

You have labels. Learn to predict the answer.
Everyday: a spam filter trained on emails marked spam or not spam.

Unsupervised

You have no labels. Find groups or patterns.
Everyday: grouping shoppers by behaviour without being told the groups.



Think *in your own project, would you have labels for your data, or just the raw data?*

Self-supervised learning: data labels itself

Hide part of the data and ask the model to fill it in. The data becomes its own answer key.

In language

"The cat sat on the ____" predict the missing word. No human labelling needed.

In vision

Blank out a patch of an image and predict it. Same trick, different data.

This is how today's largest models pretrain, and it is the bridge from learning to generating.

Both kinds of learning in your field

Environmental Science

Supervised: label land cover.
Unsupervised: group climate regions.

Materials Science

Supervised: predict strength.
Unsupervised: cluster microstructures.

Architecture

Supervised: classify styles.
Unsupervised: find design patterns.

Computer Engineering

Supervised: detect faults.
Unsupervised: flag odd sensor logs.

Chemical Engineering

Supervised: predict yield.
Unsupervised: group reaction profiles.

Use labels when you have them; let the data speak when you do not.



Part 2

Telling Apart, or Creating?

Discriminative models draw the boundary

A discriminative model learns $p(y | x)$: given an input, which label is it?

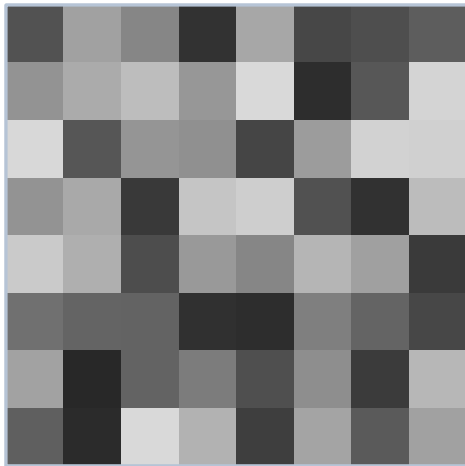


It only needs the line that separates the classes.

Cat or dog? Spam or not? It picks a side.

Everything we did in vision and NLP classification was discriminative: sort inputs into labels.

But it cannot say 'that is not real'



random noise



**forced to answer:
cat 60% / dog 40%**

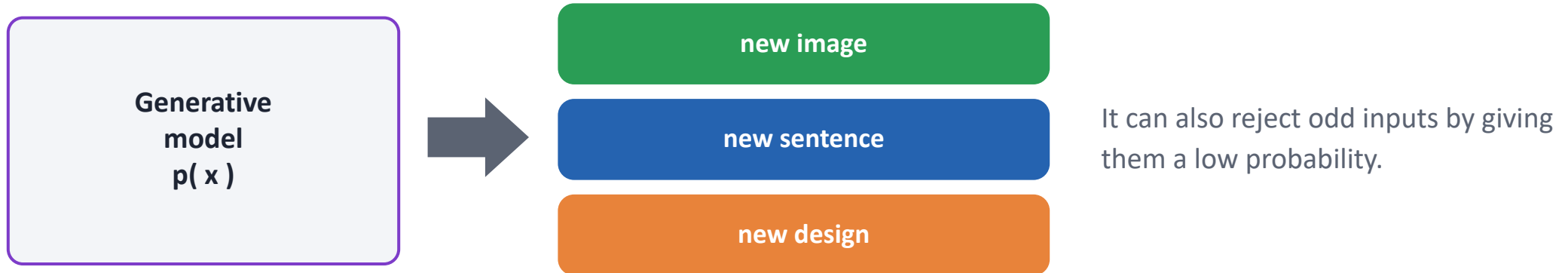
A discriminative model must label every input, even nonsense. It knows boundaries, not what real data looks like.

To know whether an input is even plausible, the model needs to learn what the data itself looks like.



Generative models learn the data itself

A generative model learns $p(x)$: what real data looks like, so it can create more of it.



Learning the whole distribution is harder, but it unlocks creating, not just sorting.



Two questions, two kinds of model

Discriminative

$$P(y | x)$$

Given this input, which label? Draws a boundary.

Generative

$$P(x)$$

How likely is this data? Can create new samples.



Think *would your task benefit more from sorting existing examples, or creating brand new ones?*



Why creating is harder than sorting

To generate, a model must understand the whole world the data lives in, not just a dividing line.

Coherence

Every pixel or word must fit together. A dog can sit or stand, but not have three heads.

Huge space

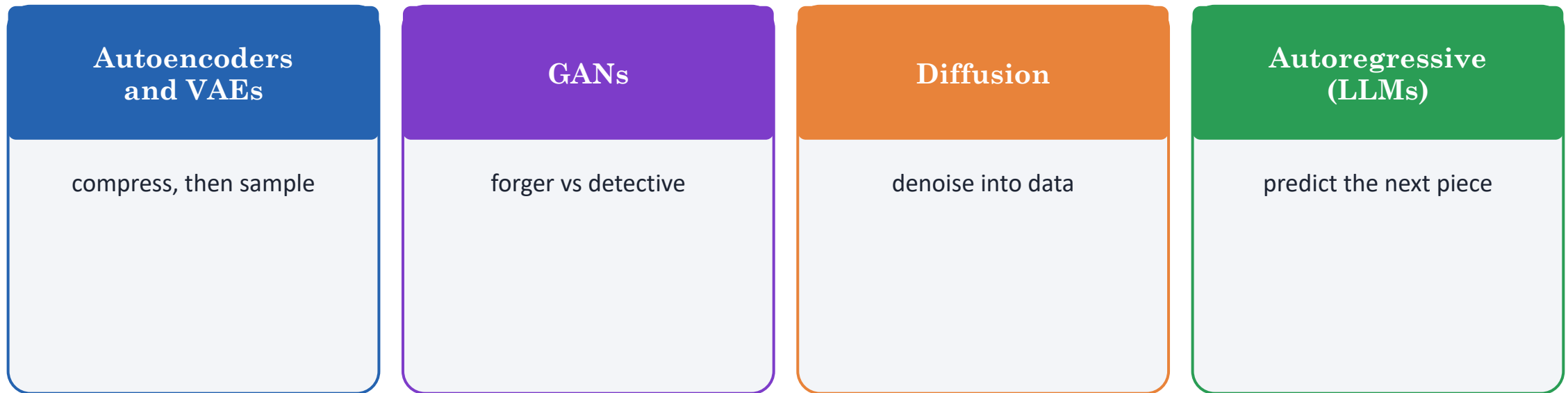
A small image has millions of pixels. Almost all combinations are nonsense.

No single answer

Many outputs are valid. The model must capture the whole range, not one point.

This is why generative models need so much data and compute: they learn the shape of an entire distribution.

A map of generative models



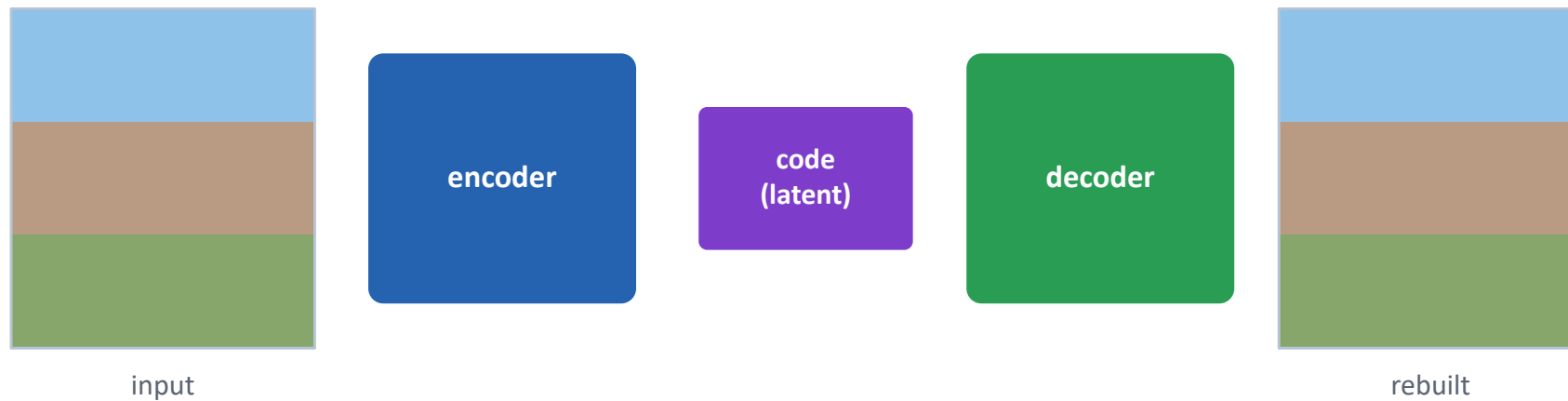
Four big families, one goal: model the data well enough to produce convincing new examples.



Part 3

The Generative Toolbox

Autoencoders: compress, then rebuild

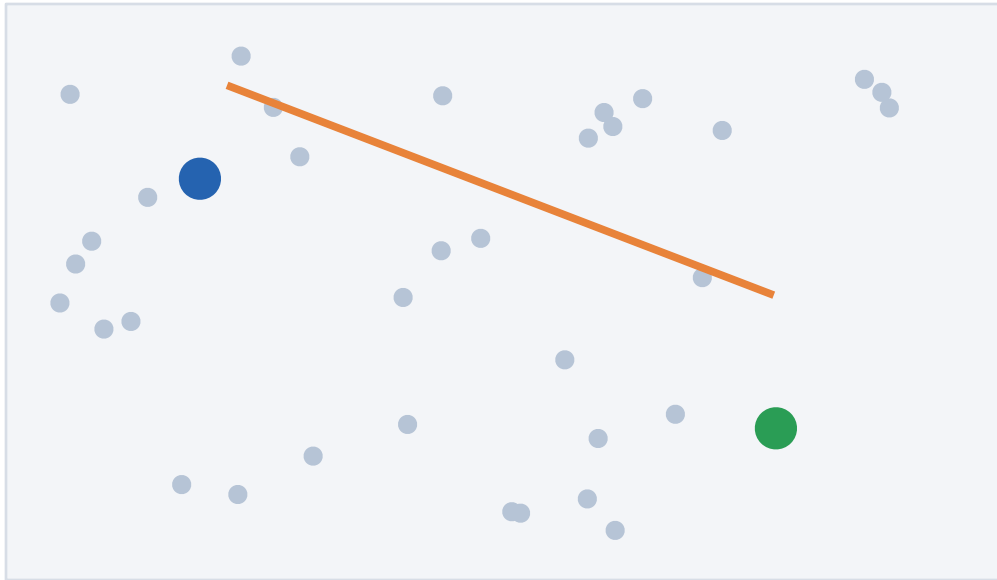


The small code must capture the essence of the data.

Squeeze an image into a tiny code, then rebuild it. The network learns the most meaningful features.

What is a latent space?

The latent space is the compressed map of codes. Each point is a possible output; nearby points are similar.



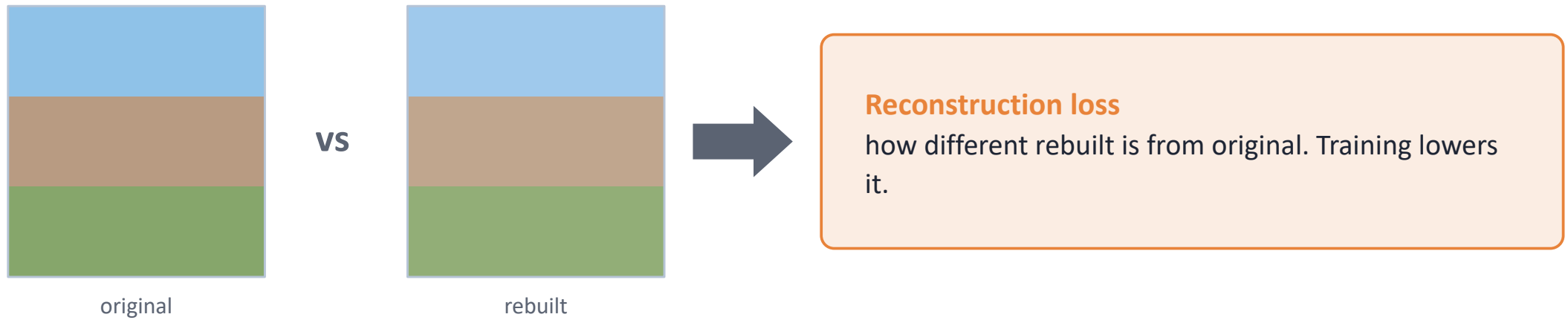
walk from one point to another and the output morphs smoothly

Move a little, change a little.

Step between two faces and you get the faces in between. The space encodes meaning, not just storage.

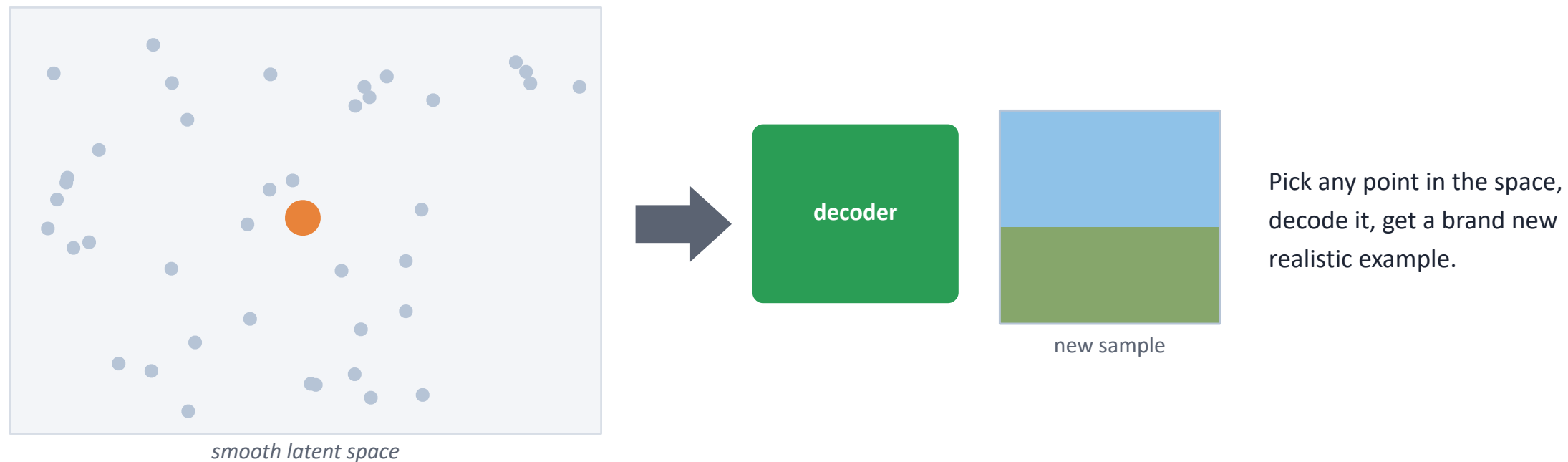
A good latent space is the key to generating: sampling a point gives a new, sensible output.

How an autoencoder learns



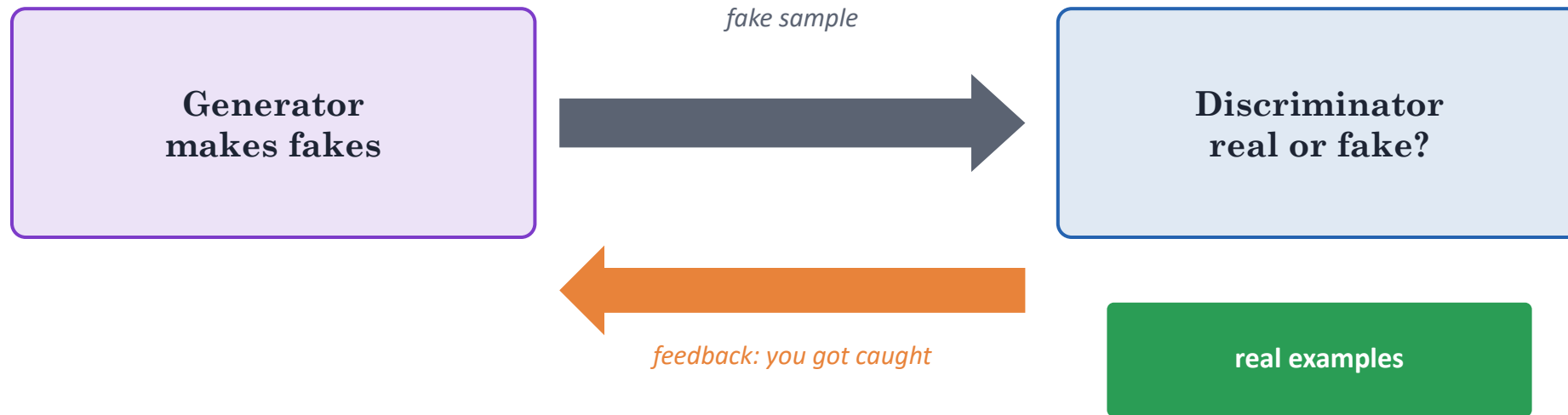
No labels needed: the input is its own target. The model learns by trying to copy it through a narrow code.

Variational autoencoders: a space you can sample



VAEs add a probabilistic twist so the latent space stays smooth. Sampling it generates new data.

GANs: a forger against a detective



Two networks train against each other. The generator keeps improving until its fakes fool the detective.



The GAN training game, and its pitfalls

When it works

Both sides improve together until fakes are hard to tell from real.

Famous for sharp, realistic faces and images.

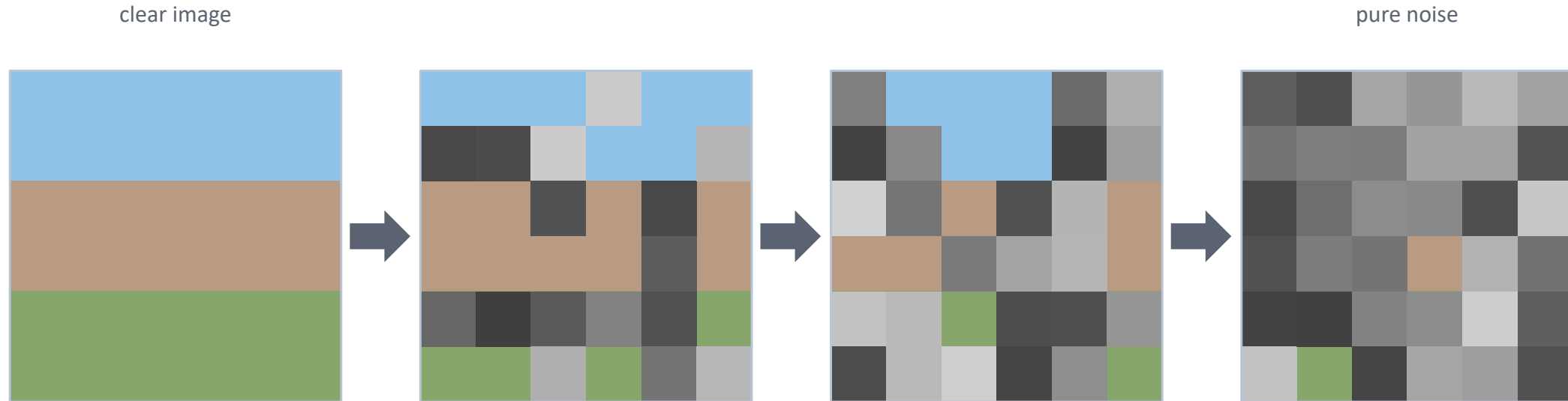
What can go wrong

Mode collapse: the generator keeps making the same few outputs.

Training can be unstable and hard to balance.

GANs are powerful but tricky to train. That is part of why diffusion models have become so popular.

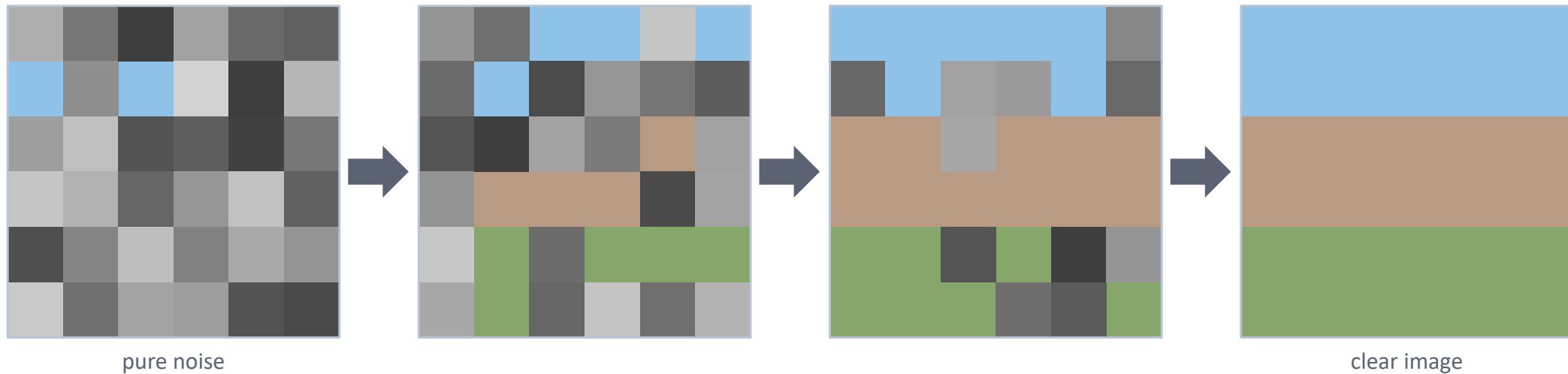
Diffusion, part 1: learn to remove noise



training: add noise step by step, and teach the model to predict the noise that was added

First we take real images and gradually destroy them with noise, recording what we added at each step.

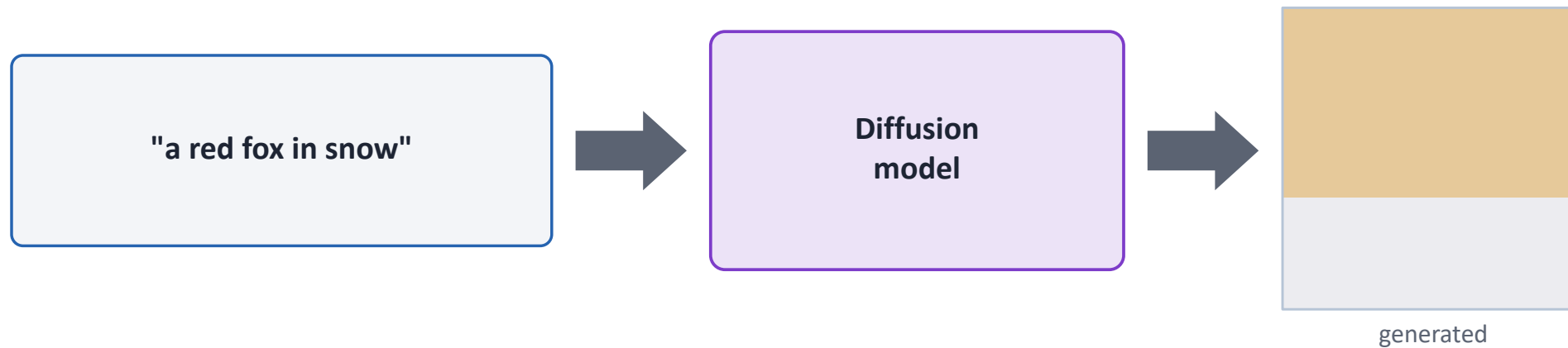
Diffusion, part 2: turn noise into a picture



generating: start from noise and remove a little at each step until an image appears

This reverse process is how modern image generators turn random noise into art.

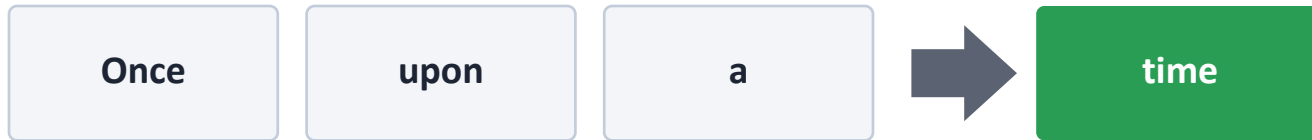
Steering generation: text to image



A prompt, sketch or label conditions the model, guiding the denoising toward what you asked for.

LLMs generate text one token at a time

Predicting the next token, over and over, is generation. That is how ChatGPT writes.



Add the new word, then predict the next one, and repeat. This is called autoregressive generation.

The Transformer and attention from Day 4 are exactly the generative model behind large language models.



Comparing the generative families

Family	How it works	Best at	Watch out for
VAE	encode, sample, decode	smooth, fast samples	slightly blurry output
GAN	forgery vs detective	sharp realistic images	unstable training
Diffusion	denoise from noise	high quality images	slow to generate
Autoregressive	predict next token	text and code	one piece at a time

No single best model. You pick the family that fits your data, your quality bar and your speed needs.



Part 4

The Bigger Picture

What generative AI can do today

1

Write and chat

2

Make images and
art

3

Generate code

4

Compose music
and voice

5

Edit and dub video

6

Draft designs and
3D

All built on the same toolbox: model the data well, then sample from it.

Generative AI in your field?

Generative AI in your field

Environmental Science

Generate synthetic satellite scenes to train models, and simulate climate scenarios.

Materials Science

Propose candidate molecules or microstructures with target properties.

Architecture

Draft design variations and floor plans from a short brief.

Computer Engineering

Generate code, test cases and synthetic sensor data.

Chemical Engineering

Suggest novel reaction routes and fill gaps in sparse datasets.

Beyond chat and art: generative models help design, simulate and fill in missing data.

Strengths and limits

Strengths

Fast first drafts of text, images, code and designs.
Explores many options quickly and fills gaps in data.

Limits

Can hallucinate, repeat bias, and cost a lot of energy.
Raises hard questions about ownership and authenticity.

A powerful assistant, not an oracle. Best used to draft and explore, with a human deciding.

Using generative AI responsibly

It can be confidently wrong

Models make up facts. Always check important claims.

It reflects its data

Bias in, bias out. Question who and what is left out.

It can mislead

Deepfakes and fakes spread fast. Verify sources.

Credit and privacy

Respect copyright and do not feed in private data.



Think before you trust a generated answer, what is one check you will always do?

A quick checklist for using it well



Check the facts

Verify names, numbers and claims against a trusted source.



Keep a human in the loop

You decide; the model drafts and suggests.



Disclose AI use

Say when content was AI assisted, and respect copyright.

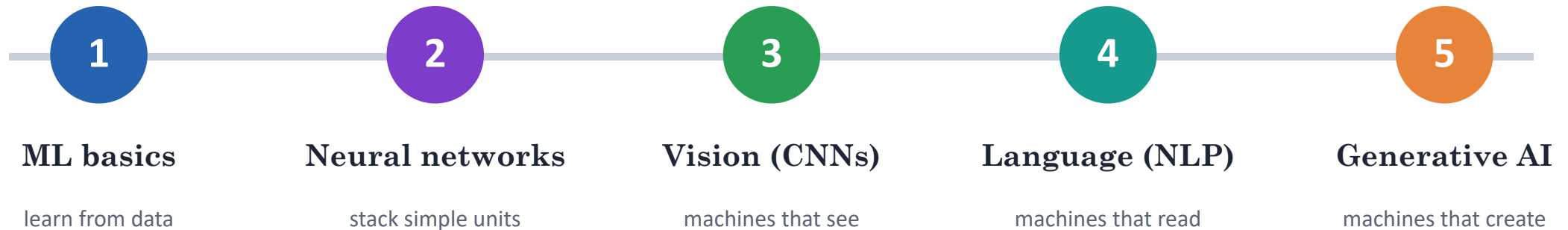


Protect data

Never paste private or sensitive data into public tools.

Treat generative AI like a fast, talented intern: useful, but its work needs checking.

The journey, in five steps



Every step built on the last: from simple patterns to systems that see, read and create.



Key takeaways

1

Supervised vs unsupervised

With labels you predict; without labels you find structure.

2

Discriminative vs generative

$p(y|x)$ draws boundaries; $p(x)$ models the data and can create.

3

The toolbox

Autoencoders, VAEs, GANs, diffusion, and autoregressive LLMs.

4

How they create

Learn a good latent space or process, then sample from it.

5

Use it responsibly

Check facts, watch for bias, respect credit and privacy.

Thank you!

You have seen machines learn to see, to read, and to create. Now go and build something. Questions?